Error handling: the 4-arg handler and async errors

10 min read · Updated 24 Jul 2026 · Topic 37 of 70

Bookmark · Mark done

Why it matters

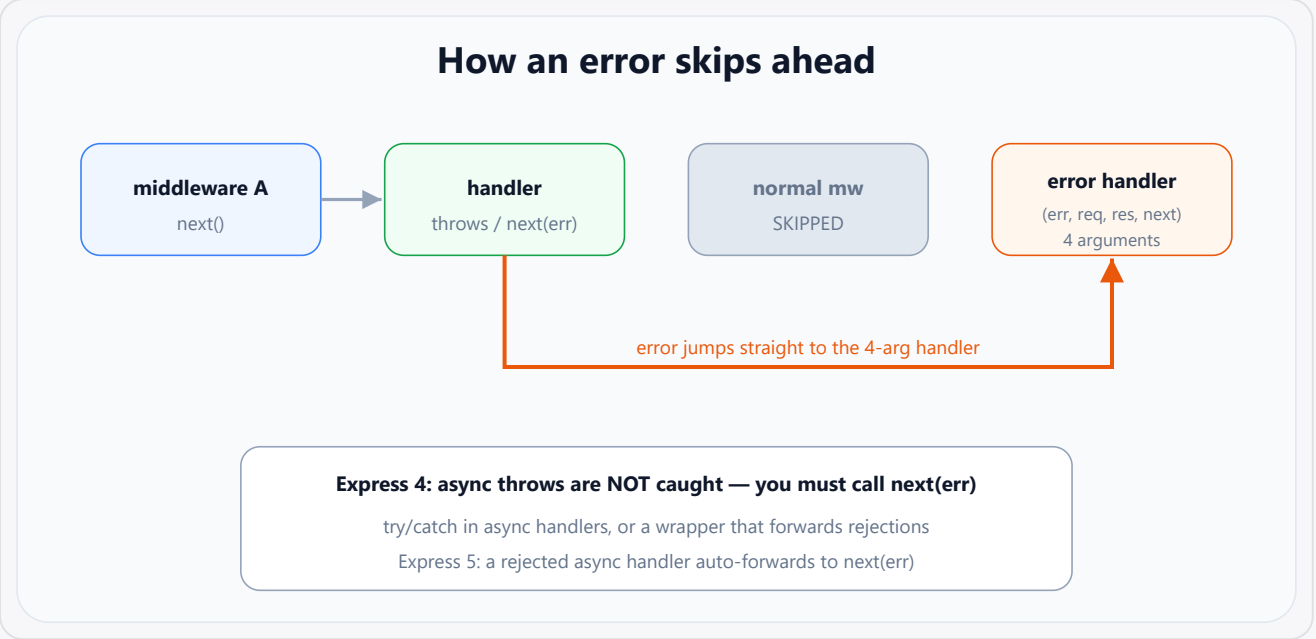
How an app handles errors is what separates a demo from a production service. Errors are not exceptional — a database is momentarily down, a client sends malformed JSON, a record is missing, a token is expired — these happen constantly. If your Express app leaks stack traces to clients, hangs on a rejected promise, or returns HTML error pages from a JSON API, it will fail review and frustrate every consumer. Interviewers love error handling because it is subtle: Express has a *specific mechanism* (a special four-argument middleware), and Express 4's most notorious footgun — async errors silently not being caught — has burned nearly every Node developer at least once.

This topic covers the whole error story: the special error-handling middleware and why it has four parameters, how errors *propagate* through the pipeline, the crucial difference between synchronous and asynchronous error handling (and how Express 5 fixes it), catching unmatched routes with a 404 handler, and centralising all of it into one place so every error in your app is formatted consistently. Get this right and your API becomes predictable under failure — which is most of what "production-ready" means.

The concept, simply

Express recognises one special kind of middleware: an **error-handling middleware has four parameters** — `(err, req, res, next)` — instead of three. That extra first parameter, `err`, is how Express identifies it. You do not call these functions directly; Express routes an error to them automatically. The rule is simple: whenever a normal middleware throws (synchronously) or calls `next(err)`, Express stops running normal

To send an error into that system you have two moves. In synchronous code you can just `throw` — Express wraps route handlers in a try/catch and turns a thrown error into `next(err)` for you. In asynchronous code (a rejected promise, an error inside a callback) you must call `next(err)` yourself in Express 4, because Express 4's try/catch cannot catch an error that happens later, after the function has already returned. Express 5 closes this gap: it awaits async handlers and forwards rejections to `next(err)` automatically.



A thrown or forwarded error skips all remaining normal middleware and jumps straight to the four-argument error handler.

Two more pieces complete the picture. A **404 handler** is a normal middleware placed *after all routes* — if a request reaches it, nothing matched, so it responds with "Not Found". And a **centralized error handler** is a single four-argument middleware placed *last*, after everything, that catches every error the app produces and turns it into one consistent response shape (status + JSON message), logging the details server-side instead of leaking them to the client.

How it works

Express identifies error middleware purely by **arity — the function's parameter count**. A function declared with four parameters is an error handler; with three (or fewer) it is

middleware and never sends errors to it. Omitting the fourth parameter is a classic silent bug — the handler simply never fires.

When an error enters the system (via `throw` in sync code or an explicit `next(err)`), Express advances its cursor past every remaining *normal* middleware and route — they are skipped entirely — and calls the next *error* middleware it finds. Inside that handler you can inspect the error, choose a status code, log it, and respond. If you call `next(err)` again from within an error handler, Express passes it to the *next* error handler down the stack; if there is none, it falls back to Express's built-in default error handler, which sends the status and, in development, the stack trace.

The synchronous-versus-asynchronous distinction is the heart of the topic. In Express 4, a route like `(req, res) => { throw new Error() }` is caught because Express wrapped the call in try/catch. But `async (req, res) => { await thatRejects() }` is *not* caught in Express 4: by the time the promise rejects, the synchronous part of the function has returned and the try/catch has already exited. The rejection becomes an unhandled promise rejection, the request hangs (no response is ever sent), and eventually the client times out. The fixes are: wrap async handlers in try/catch and call `next(err)`, use a small wrapper that attaches `.catch(next)` to the returned promise, or upgrade to Express 5 where this is handled for you.

The **default error handler** is always present at the very end of the stack, even if you write none. It sets the response status from `err.status` / `err.statusCode` (or 500), and in non-production it includes the stack. Because it runs last, your own centralized handler — placed after all routes but before Express's default takes over — is where you impose your own JSON shape and logging. Order still rules: the 404 middleware and the error handler are the last two things you register, in that order.

Code examples

1. Synchronous errors — `throw` just works.

JAVASCRIPT Copy

```
});                              // → jumps to the error handler below
```

2. Async errors — the Express 4 trap and the two fixes.

JAVASCRIPT Copy

```
// BROKEN in Express 4: the rejection is NOT caught, request hangs
app.get('/bad', async (req, res) => {
  const user = await db.find(req.params.id); // if this rejects…
  res.json(user);                            //  …this never runs, no res
});

// FIX A: try/catch and forward with next(err)
app.get('/good', async (req, res, next) => {
  try {
    const user = await db.find(req.params.id);
    res.json(user);
  } catch (err) {
    next(err);                 // hand the error to the error middleware
  }
});
```

3. A reusable async wrapper so you never write try/catch in every route.

JAVASCRIPT Copy

```
// wrap an async handler so any rejection goes to next(err)
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await db.find(req.params.id);  // rejection → next(err) au
  if (!user) throw new AppError('User not found', 404);
  res.json(user);
}));
// In Express 5 the wrapper is unnecessary — rejections auto-forward.
```

4. A 404 handler plus a centralized error handler, registered last.

JAVASCRIPT

Copy

```
// custom error type carrying a status code
class AppError extends Error {
  constructor(message, status = 500) { super(message); this.status = statu
}

// 404: reached only if no route matched (place AFTER all routes)
app.use((req, res, next) => {
  next(new AppError(`Not Found: ${req.method} ${req.originalUrl}`, 404));
});

// centralized error handler: MUST have 4 params, place LAST
app.use((err, req, res, next) => {
  const status = err.status || 500;
  if (status ≥ 500) console.error(err);        // log server-side only
  res.status(status).json({
    error: { message: status ≥ 500 ? 'Internal Server Error' : err.messag
  });
});
```

Common mistakes & gotchas

- **Writing an error handler with three parameters.** Express identifies error middleware by its four-parameter arity; drop `next` and it is treated as normal middleware and never receives errors. Declare all four even if unused.
- **Relying on `throw` inside an async handler on Express 4.** The rejection is not caught, no response is sent, the request hangs. Use try/catch + `next(err)`, an `asyncHandler` wrapper, or Express 5.
- **Placing the error handler before the routes.** It must be registered last, after all routes and the 404 handler, or the errors never reach it.
- **Leaking internals to clients.** Returning `err.message` and stack traces for 500s exposes implementation details. Log them server-side; send a generic message for 5xx.

- **Responding twice.** Handling an error and then also calling `next(err)`, or responding in both the handler and the error middleware → "headers already sent". Respond once.
- **Forgetting a 404 handler.** Without one, unmatched routes fall to Express's default, which returns a bare HTML "Cannot GET /x" — jarring for a JSON API.

Best practices

- **Use one centralized error handler** at the end of the stack that maps errors to a consistent JSON shape and status, and logs 5xx server-side.
- **Create a custom error class** (e.g. `AppError` with a `status`) so handlers can `throw new AppError('...', 404)` and the central handler reads the status.
- **Wrap async handlers** with an `asyncHandler` helper (or use `express-async-errors`, or Express 5) so no rejection ever slips past.
- **Register a 404 handler after all routes**, forwarding a 404 error into the same central handler for one uniform response shape.
- **Distinguish operational from programmer errors** — expected failures (bad input, missing record) get clean 4xx responses; unexpected bugs get a logged 500 and a generic message.
- **Never send stack traces in production;** gate verbose details behind `NODE_ENV ≠ 'production'`.

⭐ Interview lens

> **Q: "How does Express know a middleware is an error handler?"**
> A: By its arity — it has four parameters, `(err, req, res, next)`, instead of three. Express inspects the function's parameter count. If you write only three, it is treated as normal middleware and never receives errors, even if you name the first parameter `err`.

> **Q: "Why doesn't Express 4 catch errors thrown in an async route handler?"**
> A: Express 4 wraps handler calls in a synchronous try/catch. An async function

response is sent. You must call `next(err)` yourself, or use a wrapper, or Express 5 which awaits and forwards rejections automatically.

Q: "How do you handle async errors cleanly across many routes?"
A: A tiny wrapper: `const asyncHandler = fn ⇒ (req, res, next) ⇒ Promise.resolve(fn(req,res,next)).catch(next);`. Wrap each async handler with it so any rejection is forwarded to the error middleware. Alternatives are the `express-async-errors` package or upgrading to Express 5.

Q: "Where do the 404 handler and the error handler go?"
A: Both go last. The 404 handler is normal middleware after all routes — if a request reaches it, nothing matched. The error handler (four params) goes after that, dead last, so it catches everything, including the 404 you forward into it.

Q: "What does `next(err)` do differently from `next()`?"
A: `next()` continues to the next normal middleware. `next(err)` — any argument makes it an error — causes Express to skip all remaining normal middleware and route handlers and jump straight to the next four-argument error handler.

Q (trap): "Your async route occasionally hangs with no response and no error logged. Cause?"
A: Classic Express 4 async-error leak: an `await` rejected, nothing caught it, so `next(err)` was never called and no response was sent. The request hangs until timeout. Wrap the handler (try/catch or asyncHandler) or move to Express 5.

Cheat-card

- Async rejection is **NOT** caught in Express 4 → request hangs. Use `next(err)`, an `asyncHandler`, or Express 5.
- `asyncHandler`: `fn ⇒ (req,res,next) ⇒ Promise.resolve(fn(req,res,next)).catch(next)`.
- `next(err)` skips all normal middleware → jumps to error handler.
- Order: routes → **404 handler** → **error handler** (both last).
- Centralize: one handler, consistent JSON shape, log 5xx, no stack traces in prod.
- Custom `AppError` with a `status` field → central handler reads it.

QUICK CHECK — no marks, just for you.

1. How does Express recognise an error-handling middleware?

- ◯ By its arity: it has four parameters, (err, req, res, next), instead of three — you must declare all four even if next is unused
- ◯ By naming the first parameter err, regardless of parameter count
- ◯ By registering it with app.error() instead of app.use()
- ◯ By throwing inside it rather than responding

Check answer

2. Why does Express 4 fail to catch an error thrown inside an async route handler?

- ◯ Express 4 has no try/catch around handlers at all
- ◯ Express 4 wraps handlers in a synchronous try/catch, but an async function returns immediately, so by the time the promise rejects the try/catch has already exited

- Express 4 disables error handling for any handler marked async

Check answer

3. Where should the 404 handler and the centralized error handler be registered?

- Both go first, before any routes, so they can intercept everything
- The error handler goes first and the 404 handler goes last
- Both go last: the 404 handler (normal middleware) after all routes, then the four-argument error handler dead last
- They must be registered inside each route handler individually

Check answer

Previous
Middleware: the pipeline, next(), and order

Next
The req and res objects: reading and responding

LEARN MORE

ARTICLES & DOCS

- Error Handling
  expressjs.com
- Using middleware
  expressjs.com

VIDEOS

Node.js Backend Kit



REST API Tutorial (Node, Express & Mongo) #10 - Error Handling

Net Ninja

Hand-picked references for this lesson.